Q. Define FA, DFA, NFA, NFA-$\wedge$ with example.

Finite Automata : It is a mathematical model of computation used to recognize patterns.
It consists of a finite number of states and transitions between those states based on input symbols.

FA is defined as a 5-tuple :

$$FA = (Q, \Sigma, \delta, q_0, F)$$

where :

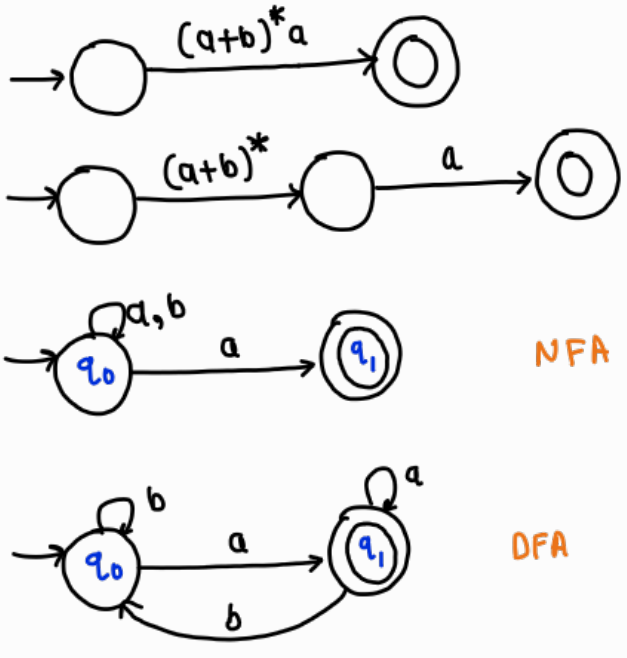
- Q = finite set of states
- Σ = input alphabet
- δ = transition function $\Rightarrow \delta : Q \times \Sigma \to Q$
- q_0 = initial state / start state
- F = set of final (accepting) states

example : FA that accepts strings ending with 'a' over $\Sigma = \{a, b\}$

Solution : $L = \{a, ba, aa, abba, aba, \ldots\}$

$RE = (a+b)^* a$

$\Rightarrow$ Transition diagram of given FA.



$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_1\}$

$\delta : Q \times \Sigma \to Q$

$\Rightarrow \delta(q_0, a) = q_1$

$\delta(q_0, b) = q_0$

$\delta(q_1, a) = q_1$

$\delta(q_1, b) = q_0$

Deterministic Finite Automata (DFA): It is a finite automata in which for every state and input symbol, there is exactly one transition.

DFA is defined as a 5-tuple:

$$FA = (Q, \Sigma, \delta, q_0, F)$$

where:

Q = finite set of states

Σ = input alphabet

δ = transition function $\rightarrow \delta : Q \times \Sigma \rightarrow Q$

q_0 = initial state / start state

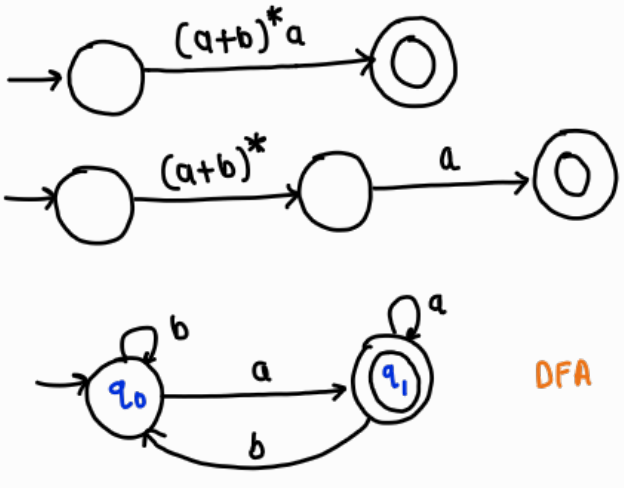
F = set of final (accepting) states

example: DFA that accepts strings ending with 'a' over $\Sigma = \{a, b\}$

solution: $L = \{a, ba, aa, abba, aba, \ldots\}$

$RE = (a+b)^* a$

$\Rightarrow$ Transition diagram of given DFA.



DFA

$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_1\}$

$\delta : Q \times \Sigma \rightarrow Q$

$\Rightarrow \delta(q_0, a) = q_1$

$\delta(q_0, b) = q_0$

$\delta(q_1, a) = q_1$

$\delta(q_1, b) = q_0$

Non-Deterministic Finite Automata (NFA):

It is a finite automata in which for every state and input symbol, there are multiple transitions for same input symbol.

NFA is defined as a 5-tuple:

$$FA = (Q, \Sigma, \delta, q_0, F)$$

where:

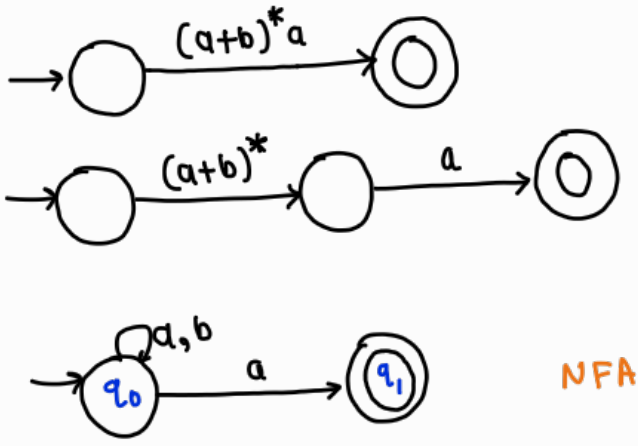
- Q = finite set of states
- Σ = input alphabet
- δ = transition function $\Rightarrow \delta : Q \times \Sigma \to 2^Q$
- q_0 = initial state / start state
- F = set of final (accepting) states

example: NFA that accepts strings ending with 'a' over $\Sigma = \{a, b\}$

Solution: $L = \{a, ba, aa, abba, aba, \ldots\}$

$RE = (a+b)^* a$

$\Rightarrow$ Transition diagram of given NFA.



NFA

$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_1\}$

$\delta : Q \times \Sigma \to 2^Q$

$\Rightarrow \delta(q_0, a) = q_0, q_1$

$\delta(q_0, b) = q_0$

$\delta(q_1, a) = \emptyset$

$\delta(q_1, b) = \emptyset$

NFA-ε : transition moves from one state to another without any symbol for input.
(called ε-moves)

NFA-ε is defined as a 5-tuple :

$FA = (Q, \Sigma, \delta, q_0, F)$

where :

Q = finite set of states

Σ = input alphabet

δ = transition function $\rightarrow \delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$

q_0 = initial state / start state

F = set of final (accepting) states

example : NFA-ε that accepts strings ending with 'a' over $\Sigma = \{a, b\}$

Solution : $L = \{a, ba, aa, abba, aba, \ldots\}$

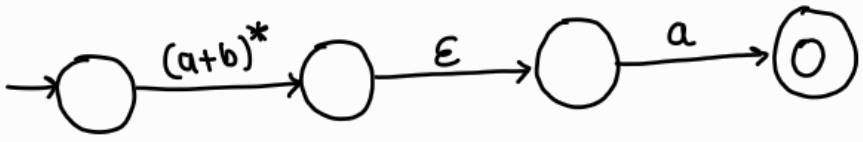
$RE = (a+b)^* a$

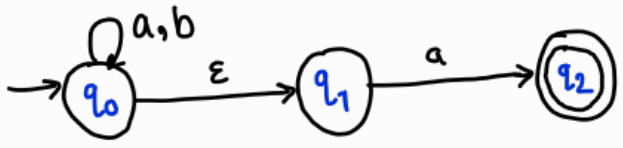
→ Break the RE :

$(a+b)^*$ → any string

then final 'a'

- separate both parts using ε-transition





$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_2\}$

$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$

Transition table :

State	a	b	ε
→q_0	q_0	q_0	q_1
q_1	q_2	∅	∅
q_2*	∅	∅	∅

Q. Recursive Definition of NFA

An NFA can be recursively constructed from regular expression by using: Base cases (symbols like $a, b, \varepsilon, \ldots$) and recursive operations (union (+), concatenation (·), *)

Rules: input symbol (for e.g. 'a') → transition $q_0 \to q_1$

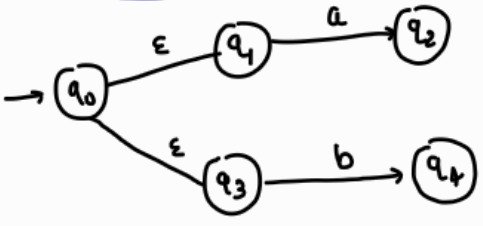
union (for e.g. $a+b$) → use ε to split in two paths

concatenation $(a \cdot b)$ → connect using ε

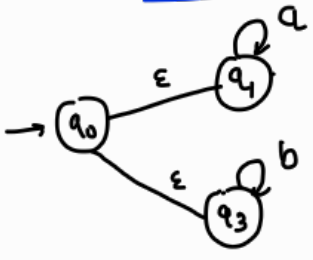
Kleene star (a^*) → add loop and ε-transitions

ex. R.E. = $(a+b)^* a$

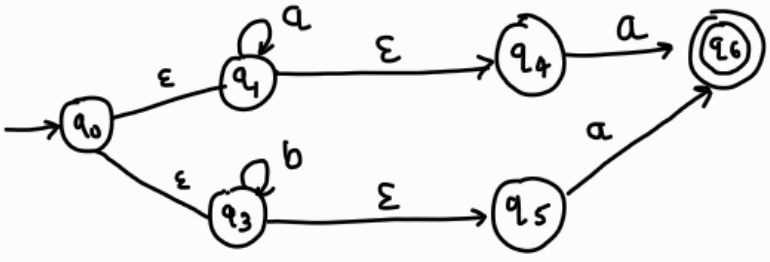
$a+b$

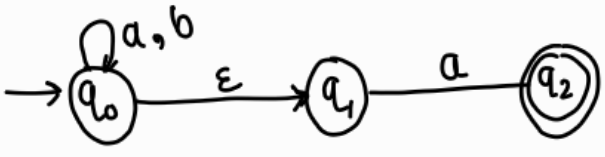


$(a+b)^*$



$(a+b)^* a$





Q. Recursive Definition of extended transition function δ^* for DFA

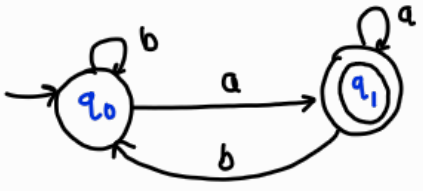
→ It gives single state reached after processing a string step by step.

Rules:

Base case : $\delta^*(q, \varepsilon) = q$

Recursive : $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$

ex. R.E. = $(a+b)^*a$



DFA

→ $\delta^*(q_0, ba)$

step 1 : start at q_0

step 2 : read 'b'

$q_0 \xrightarrow{b} q_0$

step 3 : read 'a'

$q_0 \xrightarrow{a} q_1$

$\therefore \delta^*(q_0, ba) = q_1$

Note:

- w : all characters except last one from given string
- a : last character from given string
- q : starting state
- δ : Normal transition function
- δ^* : extended transition function

$\delta(\delta^*(q, w), a)$: first process string w, then apply symbol a

$\delta^*(q, w)$: state reached after reading string w from state q.

δ (that state, a) : apply input a to that state.

In short, 1) Reach string w

2) Reach some state

3) From that state, read symbol a

Q. Recursive Definition of extended transition function δ^* for NFA.

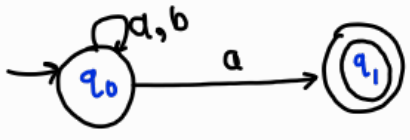
It gives set of all possible states reached after processing a string.

Rules:

Base case : $\delta^*(q, \varepsilon) = q$

Recursive : $\delta^*(q, wa) = \bigcup_{p \in \delta^*(q,w)} \delta(p, a)$

Ex. R.E. = $(a+b)^*a$



NFA

$\delta(q_0, a) = \{q_0, q_1\}$

$\delta(q_0, b) = \{q_0\}$

$\delta(q_1, a) = \emptyset$

$\delta(q_1, b) = \emptyset$

→ $\delta^*(q_0, ba)$

initial state : $\delta^*(q_0, \varepsilon) = \{q_0\}$

read first symbol : b

$\delta(q_0, b) = \{q_0\}$

read next symbol : a

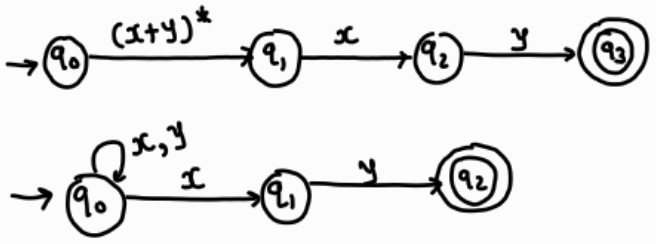
$\delta(q_0, a) = \{q_0, q_1\}$

$\therefore \delta^*(q_0, ba) = \{q_0, q_1\}$

δ^* for NFA example:

RE : $(x+y)^* xy$

Transition Diagram of NFA :



Transition function :

$\delta(q_0, x) = \{q_0, q_1\}$ $\quad \delta(q_0, y) = \{q_0\}$ $\quad \delta(q_1, y) = \{q_2\}$

$\delta(q_1, x) = \emptyset$ $\quad \delta(q_2, x) = \emptyset$ $\quad \delta(q_2, y) = \emptyset$

find $\delta^*(q_0, yyxy)$ for NFA using recursive definition.

Definition : $\delta^*(q, wa) = \bigcup_{P \in \delta^*(q,w)} \delta(P, a)$

Note :

- w : all characters except last one from given string
- a : last character from given string
- q : starting state
- P : one current state after reading w

Sol^n : string : yyxy

Break the string : $w = yyx$, $a = y$

So, $\delta^*(q_0, yyxy) = \bigcup_{P \in \delta^*(q_0, yyx)} \delta(P, y)$

① → Read first y

$\delta(q_0, y) = \{q_0\}$

$\therefore \delta^*(q_0, y) = \{q_0\}$

② → Read second y

$\delta(q_0, y) = \{q_0\}$

$\therefore \delta^*(q_0, yy) = \{q_0\}$

③ → Read x

$\delta(q_0, x) = \{q_0, q_1\}$

$\therefore \delta^*(q_0, yyx) = \{q_0, q_1\}$

④ → Read y

Now formula becomes :

$\delta^*(q_0, yyxy) = \bigcup_{P \in \{q_0, q_1\}} \delta(P, y)$

Apply y to each state of P

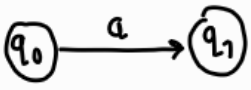
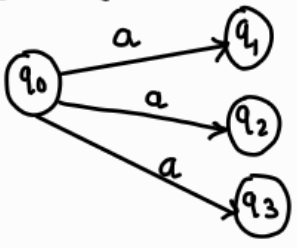
$\delta(q_0, y) = \{q_0\}$

$\delta(q_1, y) = \{q_2\}$

Take Union (U) : $\{q_0\} \cup \{q_2\}$

$\therefore \delta^*(q_0, yyxy) = \{q_0, q_2\}$

Q. DFA vs NFA

DFA	NFA
- Deterministic Finite Automata	- Non Deterministic Finite Automata
- It is defined using 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ $\delta : Q \times \Sigma \to Q$	- It is defined using 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ $\delta : Q \times \Sigma \to 2^Q$
- In case of DFA, every transition generates single state with a particular input symbol. 	- In case of NFA, transition generates more than one outcome with a particular input symbol. 
- Only one possible transition for each input symbol from any given state.	- multiple transitions for a single input symbol from a given state.
- Difficult to construct	- Easier to construct.
- All DFA are NFA	- Not all NFA are DFA
- conversion of Regular Expression to DFA is difficult.	- conversion of Regular Expression to NFA is simpler than DFA.
- Epsilon move / transition is not allowed in DFA.	- Epsilon move / transition is allowed in NFA.
- DFA requires more space.	- NFA requires less space.